



CPU Scheduler Simulator

Design, Implementation, and Analysis

Course: Operating Systems

Authors:

Oussama Laaroussi
`oussama.laaroussi@um6p.ma`

Mohamed Aymane BOURICH
`mohamed.bourich@um6p.ma`

Supervisor:

Dr. Youssef IRAQI
`youssef.iraqi@um6p.ma`

May 2, 2025

Abstract

This report details the design, implementation, and evaluation of a CPU Scheduler Simulator web application. The system provides an interactive environment for simulating various CPU scheduling algorithms such as First-Come-First-Served, Shortest Job First, Round Robin, and more. The application features real-time visualization, comparative analysis, and reports of performance metrics such as average turnaround time, waiting time, and CPU utilization. This document explores the architecture, design patterns, algorithms, and testing methodologies employed in the development of this educational tool.

Contents

1	Introduction	3
2	Project Overview	3
3	System Architecture	3
3.1	High-Level Architecture	3
3.2	Frontend Architecture	4
3.3	Backend Architecture	4
4	Design Patterns	4
4.1	Factory Pattern	4
4.2	Observer Pattern	5
4.3	Strategy Pattern	5
4.4	Store Pattern	6
5	Simulation Clock and Execution Design	6
5.1	Clock System	6
5.2	Process Execution Step Model	7
5.3	Process State Transitions	8
6	Implemented Scheduling Algorithms	8
6.1	First-Come, First-Served (FCFS)	8
6.2	Shortest Job First (SJF)	9
6.3	Round Robin (RR)	9
6.4	Priority Scheduling	10
6.5	Round Robin with Priority Scheduling	10
7	Performance Metrics	11
7.1	Turnaround Time	11
7.2	Waiting Time	11
7.3	CPU Utilization	11

7.4	Throughput	11
7.5	Visualization and Comparison	11
8	Implementation Details	11
8.1	Frontend Technologies	11
8.2	Backend Technologies	12
8.3	Key Classes and Components	12
8.3.1	Frontend Components	12
8.3.2	Backend Classes	12
8.4	Data Flow	13
9	Usage Scenarios	13
9.1	Educational Setting	13
9.2	System Design Decision Support	13
9.3	Performance Analysis	14
10	Testing and Validation	14
10.1	Probabilistic Behavior Modeling	14
10.2	Manual Testing Approach	14
10.2.1	Comparative Analysis	14
10.2.2	Edge Case Testing	15
10.2.3	Visual Validation	15
10.2.4	Long-Running Simulations	15
11	Challenges and Solutions	16
11.1	Real-time Simulation Updates	16
11.2	Algorithm Comparison Visualization	16
12	Results and Analysis	17
12.1	Performance Comparison	17
12.2	Analysis of Time Quantum in Round Robin	17
13	Future Work	17
14	Conclusion	17

1 Introduction

CPU scheduling is a fundamental concept in operating systems, determining how processes share CPU resources efficiently. Understanding the behavior and performance implications of different scheduling algorithms is crucial for both educators and system designers. The CPU Scheduler Simulator addresses this need by providing an interactive platform for experimenting with various scheduling algorithms and visualizing their effects.

2 Project Overview

The CPU Scheduler Simulator is a web-based application that allows users to:

- Create and configure processes with different arrival times, burst times, and I/O characteristics
- Select from multiple CPU scheduling algorithms
- Observe the simulation in real-time with visual representations
- Analyze performance metrics for individual algorithms
- Generate comparative reports between different scheduling policies

The primary goal is to provide an intuitive, visual understanding of how CPU scheduling algorithms work and differ from one another, serving as both an educational tool and a decision-support system for algorithm selection.

3 System Architecture

3.1 High-Level Architecture

The application follows a client-server architecture with real-time communication:

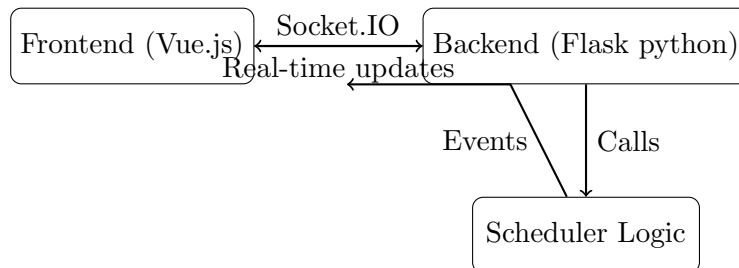


Figure 1: System Architecture

3.2 Frontend Architecture

The frontend implements a component-based architecture using Vue.js, with the following key components:

- **Process Management:** Creation and configuration of processes
- **Simulation Control:** Starting, pausing, and resetting the simulation
- **Visualization:** Gantt charts, timeline views, and state visualization
- **Comparison Report:** Comparative analysis between algorithms

State management is handled through Pinia stores, providing a centralized location for application state.

3.3 Backend Architecture

The backend implements a service-oriented architecture with:

- **Scheduler Factory:** Dynamically instantiates the appropriate scheduling algorithm
- **Scheduler Base Class:** Defines common functionality for all schedulers
- **Algorithm Implementations:** Specific scheduling algorithm logic
- **Socket.IO Service:** Handles real-time communication with the frontend
- **Metrics Calculator:** Computes performance metrics for each simulation

4 Design Patterns

The project incorporates several design patterns to ensure maintainability, extensibility, and clean architecture:

4.1 Factory Pattern

Used to create scheduler instances based on the selected algorithm type:

```
1  # Create scheduler
2      if scheduler_type == 'fcfs':
3          scheduler = FCFSScheduler(processes)
4      elif scheduler_type == 'sjf':
5          scheduler = SJFScheduler(processes)
6      elif scheduler_type == 'priority':
```

```

7         scheduler = PriorityScheduler(processes)
8         elif scheduler_type == 'rr':
9             scheduler = RoundRobinScheduler(processes,
10 time_quantum)
11         elif scheduler_type == 'priority_rr':
12             scheduler = PriorityRRScheduler(processes,
13 time_quantum)
14         else:
15             raise ValueError("Invalid scheduler type")
16
17     simulation = CPUSimulation(scheduler)

```

4.2 Observer Pattern

Implemented through Socket.IO to provide real-time updates to clients:

```

1 // Server-side
2 socket.on('compare_schedulers', () => {
3     const results = compareAllSchedulers(processes);
4     socket.emit('comparison_results', results);
5 });
6
7 // Client-side
8 socket.on('comparison_results', (data) => {
9     results.value = data;
10    createAllCharts();
11 });

```

4.3 Strategy Pattern

Each scheduling algorithm implements a common interface but provides different scheduling behavior:

```

1 class Scheduler(ABC):
2     def __init__(self, processes):
3         self.processes = sorted(processes, key=lambda p: p.
4 arrival_time)
5         self.ready_queue = []
6         self.current_time = 0
7         self.time_quantum = None
8
9     @abstractmethod
10    def next_process(self):
11        pass
12
13    def update_wait_times(self):
14        for p in self.ready_queue:
15            if p.arrival_time <= self.current_time and p.state
16 == "READY":
17                p.wait_time += 1

```

4.4 Store Pattern

Using Pinia for state management across components:

```
1 export const useSimulationStore = defineStore('simulation', {
2   state: () => ({
3     processes: [],
4     metrics: {
5       currentTime: 0,
6       currentProcess: null,
7       readyQueue: [],
8       blocked: [],
9       avg_turnaround: 0,
10      avg_waiting: 0,
11      cpu_utilization: 0,
12      total_time: 0,
13      throughput: 0,
14      total_io_blocks: 0
15    },
16    isRunning: false,
17    schedulerType: 'fcfs',
18    timeQuantum: 4,
19    history: [],
20    finalMetrics: null
21  }),
22  actions: {
23    updateMetrics(payload) {
24      // State update logic
25    }
26  }
27 });
```

5 Simulation Clock and Execution Design

5.1 Clock System

The simulator implements a discrete time-step clock system that advances in uniform increments:

- Each time unit represents a discrete step in the simulation
- The current time is maintained by the scheduler and incremented after each step
- All time-based calculations (waiting time, turnaround time, etc.) use this clock
- Process execution, I/O operations, and scheduler decisions happen within these discrete time steps

The clock system serves as the heartbeat of the simulation:

```

1 # Increment time after each simulation step
2 self.scheduler.current_time += 1

```

5.2 Process Execution Step Model

The simulation uses a step-based execution model that closely mimics real operating systems:

```

1 def step(self):
2     current_time = self.scheduler.current_time
3
4     # Update blocked processes
5     for p in self.blocked[:]:
6         if random.random() < 0.3: # 30% chance to unblock
7             p.state = "READY"
8             if isinstance(self.scheduler, (RoundRobinScheduler,
9                 PriorityRRScheduler)):
10                 self.scheduler.add(p)
11                 self.blocked.remove(p)
12             else:
13                 self.scheduler.ready_queue.append(p)
14                 self.blocked.remove(p)
15
16     # Update wait times for ready processes
17     self.scheduler.update_wait_times()
18
19     # Get next process if none running
20     if not self.current_process:
21         self.current_process = self.scheduler.next_process()
22
23     # Execute current process
24     if self.current_process:
25         result = self.current_process.execute_step(current_time)
26
27     # Process state transitions based on result...

```

Each step of the simulation involves:

1. **Unblocking I/O processes:** Processes have a probabilistic chance to complete I/O operations
2. **Updating wait times:** Incrementing wait counters for processes in the ready queue
3. **CPU scheduling:** Selecting the next process if CPU is idle
4. **Process execution:** Running a single time unit of the current process
5. **State transition:** Moving processes between ready, running, blocked, and terminated states
6. **Metrics recording:** Updating performance metrics based on process execution

- 7. **History logging:** Recording the system state for visualization

5.3 Process State Transitions

Processes in the simulator transition between four states:

- **NEW:** Process is created but not yet added to the ready queue
- **READY:** Process is in the ready queue awaiting CPU time
- **RUNNING:** Process is currently executing on the CPU
- **BLOCKED:** Process is waiting for I/O operations to complete
- **TERMINATED:** Process has completed execution

The transitions between these states are handled during each step of the simulation, with the scheduler determining which process should run next according to its scheduling algorithm.

6 Implemented Scheduling Algorithms

The system implements several classical CPU scheduling algorithms, each with its own characteristics:

6.1 First-Come, First-Served (FCFS)

A non-preemptive algorithm that processes jobs in the order they arrive. However, if the process enters I/O blocking state, the CPU proceed to run the next in the queue. Once the process become ready, it is added to the end of the queue.

Algorithm 1 FCFS Algorithm

```
1: Initialize empty ready queue
2: while there are processes to schedule do
3:   Add newly arrived processes to ready queue
4:   if CPU is idle and ready queue is not empty then
5:     Dequeue first process from ready queue
6:     Assign CPU to this process
7:   end if
8:   Advance time
9: end while
```

6.2 Shortest Job First (SJF)

A non-preemptive algorithm that selects the process with the shortest burst time. However, if the process enters I/O blocking state, the CPU proceed to run the next in the queue. Once the process become ready, it is added to the queue based on its remaining execution time.

Algorithm 2 SJF Algorithm

```
1: Initialize empty ready queue
2: while there are processes to schedule do
3:   Add newly arrived processes to ready queue
4:   if CPU is idle and ready queue is not empty then
5:     Find process with shortest remaining time in ready queue, initially
       the remaining time is the burst time
6:     Assign CPU to this process
7:   end if
8:   Advance time
9: end while
```

6.3 Round Robin (RR)

A preemptive algorithm that assigns a fixed time quantum to each process.

Algorithm 3 Round Robin Algorithm

```
1: Initialize empty ready queue
2: Set time quantum  $q$ 
3: while there are processes to schedule do
4:   Add newly arrived processes to ready queue
5:   if CPU is idle and ready queue is not empty then
6:     Dequeue first process from ready queue
7:     Assign CPU to this process for up to  $q$  time units
8:     if process completes then
9:       Mark as completed
10:    else
11:      Add process back to end of ready queue
12:    end if
13:  end if
14:  Advance time
15: end while
```

6.4 Priority Scheduling

Processes are scheduled according to their priority values.

Algorithm 4 Priority Scheduling Algorithm

```
1: Initialize empty ready queue
2: while there are processes to schedule do
3:   Add newly arrived processes to ready queue
4:   if CPU is idle and ready queue is not empty then
5:     Find process with highest priority
6:     Assign CPU to this process until completion
7:   end if
8:   Advance time
9: end while
```

6.5 Round Robin with Priority Scheduling

A preemptive algorithm that assigns a fixed time quantum to each process with considering it's priority. Priority Queues were used for a better time complexity of $O(\log(n))$ where n is the number of processes.

Algorithm 5 Round Robin with Priority Scheduling Algorithm

```
1: Initialize empty ready Priority Queue
2: while there are processes to schedule do
3:   Add newly arrived processes to ready Priority Queue
4:   if CPU is idle and ready queue is not empty then
5:     Find process with highest priority and follow the order of the
       queue in the priority queue.
6:     Assign CPU to this process for up to q time units afterwards ,
       push it to the end of the priority Queue.
7:   end if
8:   Advance time
9: end while
```

7 Performance Metrics

The system calculates several key performance metrics to evaluate and compare scheduling algorithms:

7.1 Turnaround Time

The time from process submission to completion.

$$\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$$

7.2 Waiting Time

The time a process spends waiting in the ready queue.

$$\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$$

7.3 CPU Utilization

The percentage of time the CPU is busy.

$$\text{CPU Utilization} = \frac{\text{Total Busy Time}}{\text{Total Time}} \times 100\%$$

7.4 Throughput

The number of processes completed per unit of time.

$$\text{Throughput} = \frac{\text{Number of Processes}}{\text{Total Time}}$$

7.5 Visualization and Comparison

The application provides visual comparisons using various chart types:

- Bar charts for turnaround time, waiting time, and total execution time
- Radar charts for CPU utilization
- Gantt charts for process execution timeline

8 Implementation Details

8.1 Frontend Technologies

- **Vue.js**: Component-based UI framework
- **Pinia**: State management library

- **Chart.js:** Chart visualization library
- **Socket.IO Client:** Real-time communication
- **Tailwind CSS:** Utility-first CSS framework

8.2 Backend Technologies

- **Python:** Python Programming Language
- **Flask:** Python Web framework
- **Socket.IO:** Real-time bidirectional communication

8.3 Key Classes and Components

8.3.1 Frontend Components

- **ProcessForm:** Allows users to create and configure processes
- **ProcessList:** Displays and manages the list of created processes
- **SimulationControl:** Controls for running/pausing/resetting the simulation
- **GanttChart:** Visualizes the execution timeline of processes
- **MetricsDisplay:** Shows real-time performance metrics
- **ComparisonReport:** Compares metrics across different algorithms

8.3.2 Backend Classes

- **Scheduler:** Abstract base class for all schedulers
- **FCFSScheduler:** First-Come-First-Served implementation
- **SJFScheduler:** Shortest Job First implementation
- **RoundRobinScheduler:** Round Robin implementation
- **PriorityRRScheduler:** Round Robin with Priority implementation
- **PriorityScheduler:** Priority scheduling implementation
- **Process:** Represents a process with its attributes
- **ProcessManager:** Manage loading and saving processes
- **CPUSimulation:** Manage running the Scheduler and advancing time and managing blocked processes

8.4 Data Flow

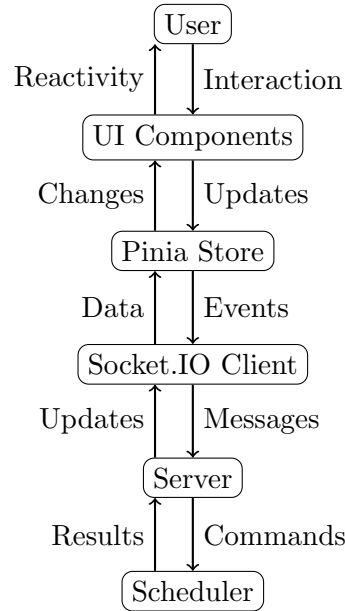


Figure 2: Data Flow Diagram

9 Usage Scenarios

The CPU Scheduler Simulator serves several usage scenarios:

9.1 Educational Setting

- Demonstrate different scheduling algorithms in real-time
- Create custom process sets to highlight specific algorithm behaviors
- Show the impact of process characteristics on scheduling performance
- Generate comparison reports for classroom discussion

9.2 System Design Decision Support

- Test different scheduling policies on simulated workloads
- Compare performance metrics to select the most appropriate algorithm
- Experiment with time quantum values for Round Robin scheduling
- Optimize for specific metrics based on system requirements

9.3 Performance Analysis

- Evaluate algorithm behavior under various process mixes
- Identify bottlenecks in scheduling for specific workloads
- Generate detailed PDF reports for documentation
- Study the impact of I/O operations on scheduling efficiency

10 Testing and Validation

10.1 Probabilistic Behavior Modeling

A key feature of the simulator is its modeling of probabilistic I/O behavior, which mimics the unpredictable nature of real-world computing systems:

```
1 # Update blocked processes with probabilistic behavior
2 for p in self.blocked[:]:
3     if random.random() < 0.3: # 30% chance to unblock
4         p.state = "READY"
5         # Return process to appropriate queue...
```

This probabilistic approach:

- Models the unpredictable duration of I/O operations in real systems
- Creates varied execution patterns between simulation runs
- Produces more realistic CPU utilization patterns
- Tests the adaptability of scheduling algorithms to changing conditions

The 30% probability parameter can be adjusted to model different types of systems, with lower values representing I/O-bound workloads and higher values representing CPU-bound workloads.

10.2 Manual Testing Approach

Testing was conducted manually through several approaches:

10.2.1 Comparative Analysis

For simple process sets with deterministic behavior (no I/O), results were manually calculated and compared with simulation output. This provided a baseline validation of the scheduling algorithms' implementation.

10.2.2 Edge Case Testing

The simulator was tested with various edge cases:

- Empty process sets
- Single process execution
- Processes with zero burst time
- Processes with identical arrival times
- Processes with excessive I/O operations
- Very large time quantum values in Round Robin

10.2.3 Visual Validation

The Gantt chart visualization was used to manually verify the scheduler behavior:

- Correct process selection order
- Proper handling of preemption
- Accurate representation of process states
- Consistent time tracking across the simulation

10.2.4 Long-Running Simulations

Extended simulations with large process sets were run to evaluate:

- System stability over time
- Memory usage patterns
- Performance under high load
- Consistency of metrics calculation

Manual testing proved effective for validating both the correctness of the scheduling algorithms and the realistic behavior of the probabilistic elements in the simulation.

11 Challenges and Solutions

11.1 Real-time Simulation Updates

Challenge: Ensuring smooth real-time updates without overwhelming the browser.

Solution: Implemented throttling of updates and optimized state management:

```
1 // Server-side
2 const sendUpdate = throttle((socket, data) => {
3   socket.emit('simulation_update', data);
4 }, 100);
5
6 // Client-side
7 socket.on('simulation_update', (data) => {
8   store.updateMetrics(data);
9 });
```

11.2 Algorithm Comparison Visualization

Challenge: Creating meaningful visual comparisons between algorithms.

Solution: Used different chart types optimized for each metric and implemented a recommendation engine:

```
1 const getRecommendation = () => {
2   let scores = {};
3   for (const algo in results.value) {
4     scores[algo] = 0;
5
6     // Lower is better for turnaround and waiting
7     if (getBestAlgorithm('avg_turnaround', true) === algo)
8       scores[algo] += 3;
9     if (getBestAlgorithm('avg_waiting', true) === algo)
10      scores[algo] += 3;
11
12    // Higher is better for CPU utilization
13    if (getBestAlgorithm('cpu_utilization') === algo)
14      scores[algo] += 2;
15
16    // Lower is better for total time
17    if (getBestAlgorithm('total_time', true) === algo)
18      scores[algo] += 2;
19  }
20
21  const bestAlgo = Object.keys(scores).reduce((a, b) =>
22    scores[a] > scores[b] ? a : b
23  );
24
25  return `Based on the simulation results: ${bestAlgo} provides
26    the best overall performance...`;
27 };
```

12 Results and Analysis

12.1 Performance Comparison

Different algorithms excel in different scenarios:

- **FCFS:** Simple implementation but can lead to long waiting times if short processes arrive after long ones.
- **SJF:** Optimal for minimizing average waiting time but requires knowing burst times in advance.
- **RR:** Good for interactive systems, ensuring fair CPU distribution but with moderate waiting times.
- **RR with Priority:** Good for interactive systems with priorities, ensuring priority policy but with moderate waiting times.
- **Priority:** Effective for systems where process importance varies, but can lead to starvation.

12.2 Analysis of Time Quantum in Round Robin

The time quantum value significantly impacts Round Robin performance:

- **Small quantum:** More responsive but higher context-switching overhead
- **Large quantum:** Less overhead but approaches FCFS behavior

13 Future Work

Several enhancements could be made to the simulator:

- **Additional Visualization Tool:** Add visualization for other metrics and enable more personalized run of the scheduler.

14 Conclusion

The CPU Scheduler Simulator provides an interactive, visual environment for understanding and comparing different CPU scheduling algorithms. Through careful implementation of software design patterns and a component-based architecture, the system offers a flexible platform for educational and analytical purposes.

The comparison capabilities highlight the trade-offs inherent in scheduler selection, with different algorithms optimizing for different metrics. The visualization tools make it easy to communicate these differences and support informed decision-making.